

The Tech Stack, Explained

Fairmarket eCommerce — a plain-language guide for a new developer or IDE

Prepared 16 September 2026

This is a plain-language walkthrough of every major piece of technology this project is built on: what each one actually is, why it was chosen over the realistic alternatives, and what the jargon words attached to it mean. Written for someone who can use a computer well but hasn't necessarily built a website like this before.

The Big Picture, in One Paragraph

Fairmarket is a website (built to also support a future mobile app without rework). It is built with Next.js, a tool for building websites using the React library and the JavaScript/TypeScript programming language. All of the site's data — user accounts, products, orders, messages — lives in a hosted database service called Supabase, which is really just a managed version of a very well-known database engine called Postgres, with a login system and file storage bundled in. Payments are handled by Paystack, a Nigerian payment company, so the site never has to touch a customer's card details directly. The whole site is hosted on Vercel, a service built specifically to run Next.js sites. That's the entire stack — five main pieces, each doing one job, each replaceable on its own if it were ever needed.

Each Piece, and Why It Was Picked Over the Alternatives

1. Next.js (with React) — the framework the whole site is built with

What it is: a set of tools for building a website where some pages are generated on the server (for speed and for search engines to read easily) and some parts update instantly in the browser (for a smooth, app-like feel) — without having to build two separate systems for those two things. It is built on top of React, the most widely used library for building interactive web pages, made by Meta (Facebook).

Why this instead of the alternatives:

- vs. a plain React app (no Next.js): plain React alone doesn't generate pages on the server, which matters a lot for a shopping site — product pages need to be readable by Google and load fast on a slow connection, which server-generated pages do much better than a pure "everything happens in the browser" app.
- vs. WordPress/PHP (what the BookFlow plugin project uses): WordPress is excellent for content-driven sites and has a huge plugin ecosystem, but a real marketplace with custom checkout, seller dashboards, negotiation, and a state machine for orders is easier to build and reason about as custom application code than by bending a CMS to do all of that.
- vs. a totally separate backend (e.g. a Django or Node/Express API server plus a separate frontend): that's two codebases to build, deploy, and keep in sync. Next.js lets one codebase serve both the pages people see and the small backend endpoints (like the payment webhook) the site needs, which is simpler for a small team.

2. TypeScript — the programming language

What it is: JavaScript (the language every web browser understands) with an extra layer that catches a whole category of mistakes — like accidentally treating a product's price as text instead of a number — while the code is being written, instead of a customer discovering the bug later.

Why this instead of plain JavaScript: for a project with orders, payments, and money involved, catching "you used the wrong shape of data here" mistakes before the code ever runs is worth the small amount of extra upfront structure it asks for.

3. Tailwind CSS — how the site is styled/designed

What it is: a way of styling a webpage by adding small, pre-named building-block labels directly onto each element (e.g. "make this bold and blue") instead of writing separate style rules in a different file.

Why this instead of the alternatives:

- vs. writing plain CSS by hand: plain CSS is more flexible but much easier to end up with inconsistent spacing/colors across a large site as more pages get added over time.
- vs. a component library like Bootstrap: Bootstrap ships its own pre-made look, which is faster to start with but harder to make look like your own distinct brand rather than "a Bootstrap site." Tailwind is closer to the metal — more setup, but the result looks custom rather than templated.

4. Supabase — the database, login system, and file storage

What it is: three closely related pieces bundled as one hosted service:

- The database: a hosted, managed copy of Postgres — one of the most trusted, widely used database engines in the world, free and open-source, run here as a paid hosted service so nobody has to manage a database server themselves.
- Auth (login/sign-up): handles passwords, sessions, and "who is currently logged in," so the project didn't have to build password security and session handling from scratch.
- Storage: where uploaded images (product photos, store logos, ID verification documents) actually live, similar in concept to Google Drive or Dropbox but built into the same service as the database.

Why this instead of the alternatives:

- vs. Firebase (Google's similar all-in-one product): Firebase's database is a different kind (documents, not the row-and-column tables Postgres uses) which is a worse fit for a marketplace's naturally relational data — sellers have products, products have categories, orders reference both a buyer and a seller, and so on. Postgres handles those relationships and enforces data correctness directly in the database itself.
- vs. running your own Postgres server (e.g. on a rented server): fully possible, but means someone has to handle backups, security patches, and scaling by hand. Supabase does that maintenance as part of the service.
- A specific Postgres feature this project leans on heavily: "Row Level Security" (RLS) — rules written directly into the database itself saying exactly who is allowed to see or change which row of data (e.g. "a seller can only edit their own products, never anyone else's"), enforced by the database no matter what path the request came through. This is a major reason Postgres/Supabase was chosen — it moves a lot of "is this person allowed to do this?" security logic into the one place it can't accidentally be bypassed.

5. Paystack — payments

What it is: a Nigerian payment-processing company (similar in role to Stripe, which is very well known internationally but has weaker direct support for Nigerian bank cards/transfers). Paystack is

the part of the system that actually touches a customer's card or bank details — the site itself never stores or sees that information directly, which is both safer and avoids needing to meet the strict security certifications required to handle card numbers directly.

Why this instead of the alternatives: Paystack is the dominant processor for Nigerian bank cards and bank transfers, and supports splitting a payment between multiple recipients — useful for a marketplace that may eventually pay sellers directly out of each sale. Stripe or Flutterwave were the realistic alternatives; Paystack was chosen for its strength specifically in the Nigerian market this project targets first.

6. Vercel — hosting (where the live website actually runs)

What it is: the company that makes Next.js also runs a hosting service built specifically to run Next.js sites with minimal manual setup — push code, it goes live automatically, including a temporary "preview" website for every change before it's approved.

Why this instead of the alternatives: a traditional rented server (a "VPS") or a general cloud platform like AWS can run a Next.js site too, but both need someone to manually configure how the server runs, restarts, and scales. Vercel does all of that automatically for the exact framework this project uses, which trades a small amount of control for a large amount of saved setup time.

7. Testing tools — Vitest and Playwright

What they are: two different kinds of automated "does this still work?" checks.

- Vitest: runs small, fast checks against individual pieces of logic (e.g. "does the fair-price calculation give the right answer for these numbers?") — 240+ of these currently run in seconds and don't need a real browser or database.
- Playwright: drives an actual browser through a real user journey (e.g. "sign up, list a product, buy it, confirm the order arrives") to catch problems that only show up when all the pieces work together.

Why these instead of Jest (a very common alternative to Vitest): Vitest does the same job as Jest but starts up faster and works more smoothly with this project's specific build tools — a practical choice more than a philosophical one.

Jargon Glossary

Plain explanations of the specific technical words that come up most often in this codebase and its documentation.

npm

Short for "Node Package Manager." Almost every JavaScript/TypeScript project is built from hundreds of small, reusable, free code libraries written by other people (e.g. the library that handles dates, or the library that talks to Paystack). npm is the tool that downloads and manages all of those libraries for a project. The file `package.json` (visible at the root of this project) is the official list of exactly which libraries, and which versions, the project depends on. Running "npm install" reads that list and downloads everything on it.

Prisma (not used in this project, but a very common tool worth knowing)

Prisma is a popular tool (called an "ORM" — see below) that lets you read and write to a database using plain TypeScript/JavaScript instead of writing raw database query language (SQL) by hand. This project does not use Prisma — it talks to the Supabase/Postgres database using Supabase's own official connector library instead, writing real SQL directly for the database structure (in the `supabase/migrations` folder) and using Supabase's query-building library for reading and writing data

day to day. This was a deliberate choice, not an oversight: Postgres's "Row Level Security" feature (explained above) is easiest to rely on correctly when the queries are close to the real SQL, and Supabase's own tooling is built specifically to work with RLS.

ORM (Object-Relational Mapper)

The general category of tool that Prisma belongs to — software that translates between "database rows and tables" and "objects in your programming language," so a developer can write more familiar code instead of raw database query language. Not every project needs one; this one deliberately works closer to the database directly, as explained above.

API / Route Handler

API stands for "Application Programming Interface" — in this context, a specific web address the site's own server responds to that isn't a page a person looks at, but a small task another piece of software asks it to do (e.g. "here's a payment confirmation from Paystack, please record it"). In this project's Next.js code, these live under `src/app/api/` and are called "route handlers."

Migration

A small, numbered, saved file recording one specific change made to the database's structure over time (e.g. "add a `phone_number` column to the `seller_profiles` table"). Every migration this project has ever run is saved, in order, in `supabase/migrations/` — which means the exact structure of the database can always be rebuilt from scratch just by running every file in that folder in order, which is exactly how a brand-new copy of this database (for a fresh Supabase project, mentioned in the pending-tasks document) would be created.

RLS (Row Level Security)

A Postgres database feature (explained above under Supabase) where the rule "who can see or change this specific piece of data" is written into the database itself, not just into the website's code — so even a bug in the website code can't accidentally show one seller another seller's private data, because the database itself would refuse the request.

SSR / CSR (Server-Side vs. Client-Side Rendering)

Two different ways a webpage can be put together. Server-side means the finished, ready-to-read page is built on Fairmarket's own server and sent to the visitor's browser already complete — good for speed and for search engines. Client-side means the page is assembled inside the visitor's own browser after it arrives, using JavaScript — good for parts of a page that need to feel instant and app-like after the initial load. Next.js is specifically valuable because it lets a single page use both approaches for different parts of itself.

Environment variables (.env)

Settings and secret values (like the Paystack secret key) that the code needs to run, but that should never be typed directly into the code itself — partly for security (so a secret key is never accidentally shared alongside the code), and partly so the same code can run with different settings in different places (a developer's own computer vs. the real, live website). The file `.env.example` lists every setting the project expects, without any real secret values filled in, as a template.

CI (Continuous Integration)

An automatic checklist that runs every time new code is about to be added to the project — in this case, running all of the Vitest and database tests automatically and refusing the change if any of them fail. This project's checklist is written and ready (`.github/workflows/ci.yml`) but has never actually run for real yet, because doing so requires the code to live on a real GitHub repository, which it does not yet (see the Pending Tasks document).

Linting (ESLint)

An automatic style-and-mistake checker that scans the code for common error patterns and inconsistent formatting, without actually running the code — catches things like an unused variable or a likely typo before they ever become a real bug.

Git / commit / repository

Git is the system that tracks every change ever made to this project's code, one saved snapshot ("commit") at a time, so any change can always be reviewed or undone. A "repository" (or "repo") is the full folder of code plus its entire saved history. This project's repository currently exists only on this computer — pushing it to a hosting service like GitHub (mentioned in the Pending Tasks document) is what would let CI run, let other developers access it, and back it up somewhere other than one machine.